

Differential Drive Mobile Robot Simulation and Dead Reckoning

Pravin Oli (u1999699), and Gebrecherkos Gebreslassie (u1999542)

October 21, 2024

Abstract

This report presents the simulation and analysis of pose and trajectory control in a Differential Drive Mobile Robot. The pre-lab session involved computation of frame transformations by defining the robot's 3 DoF pose and performing direct and inverse compounding operations using $(\oplus)$ and $(\ominus)$ operators. The lab focused on simulating circular and figure-8 trajectories using the `SimulatedRobot` and `DifferentialDriveSimulatedRobot` classes to study localization and mapping. Key tasks included completing code for reading encoders, compass data, and visualizing the robot's movement. Dead reckoning localization, incorporating Gaussian noise, estimated the robot's position from wheel encoder readings. Post-lab simulations revealed the effects of noise on pose updates in trajectory, emphasizing the need for managing sensor noise to minimize errors in dead reckoning.

1 Introduction

The *Pre-Lab* session was about defining a robot pose in 3 Degree of Freedom (DoF) as well as composing different poses of robot using the direct and the inverse compounding operations. The direct and inverse compounding operation was done by using plus $(\oplus)$ and minus $(\ominus)$ operators. Two transformations of robot pose ie. from A-Frame to B Frame and from B-Frame to C-Frame respectively was defined and final pose was computed using plus operator and vice-versa for inverse operation using minus operator

$$\begin{aligned}\mathbf{A}_{\mathbf{x}_B} &= [{}^A x_B \quad {}^A y_B \quad {}^A z_B \quad {}^A \psi_B]^T \\ \mathbf{B}_{\mathbf{x}_C} &= [{}^B x_C \quad {}^B y_C \quad {}^B z_C \quad {}^B \psi_C]^T \\ \mathbf{A}_{\mathbf{x}_C} &= \mathbf{A}_{\mathbf{x}_B} \oplus \mathbf{B}_{\mathbf{x}_C} = [{}^A x_C \quad {}^A y_C \quad {}^A z_C \quad {}^A \psi_C]^T \\ \mathbf{B}_{\mathbf{x}_C} &= \mathbf{A}_{\mathbf{x}_C} \ominus \mathbf{A}_{\mathbf{x}_B} = [{}^B x_C \quad {}^B y_C \quad {}^B z_C \quad {}^B \psi_C]^T\end{aligned}$$

The *Lab session* was about simulating the movement of differential drive robot in different trajectory viz. circular path and eight shape paths. The modelling of robot simulation

was performed using `SimulatedRobot` and `DifferentialDriveSimulatedRobot` classes to study the localization and mapping algorithms of robotic systems within a virtual environment.

The *Post Lab session* includes the missing part of code to be written and simulation of robot on circular and eight trajectory and finally comparing it with dead reckoning using noise.

The contents in this reports is organized as follows: Section 2 describes how to simulate the robot by implementing the *DifferentialDriveSimulatedRobot* class inheriting from the *SimulatedRobot* base class. This section also explains how to use the class to simulate the trajectories requested in the lab. Section 3 details the dead reckoning localization method of a differential drive mobile robot. The technique is implemented in the *DR_3DOFDifferentialDrive* class as a child class of the *Localization* class providing an implementation for their virtual methods. Finally, the section 4 is devoted to the description of the problems founds during the lab.

2 Simulation

The simulated class robot is generic robot model with attributes for motion, sensor simulation, and visualization. It is designed to be extensible for creating specialized simulations. The `DifferentialDriveSimulatedRobot` class extends this by focusing on differential drive robots, which use two independently controlled wheels. It overrides methods from the base class to accurately simulate the unique motion and sensors of such robots, like wheel encoders and compasses, ensuring precise parameterized.

In the code file `DifferentialDriveSimulatedRobot.py`, we completed the missing code for the following function: `ReadEncoders` and `ReadCompass`

- The `ReadEncoders` method measures wheel distances and converts them into pulse counts based on robot velocities and wheel parameters.
- The `ReadCompass` method provides a noisy yaw reading for the robot's orientation.
- The `PlotRobot` method updates the robot's visual representation in real-time, reflecting its current position and orientation in the environment.

In the output the simulated trajectory of Differential Drive Robot in circular and Eight shaped path was obtained. Figure 1 shows the simulated result for circular path and Figure 2 shows simulated result for Eight shaped path

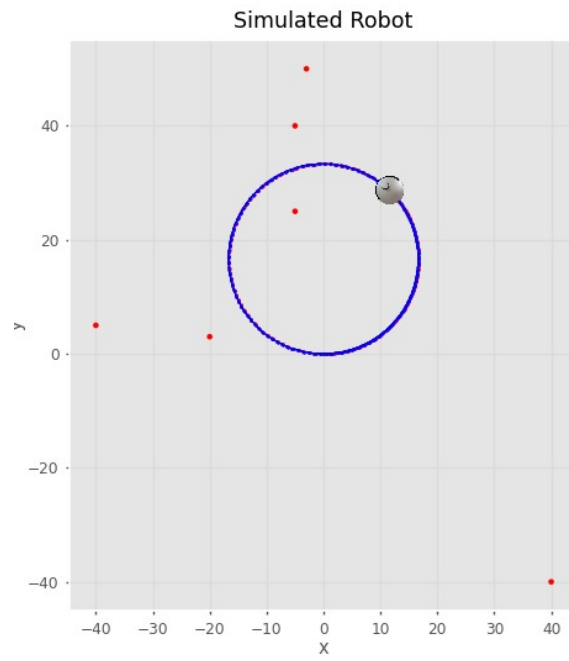


Figure 1: simulation for circular trajectory.

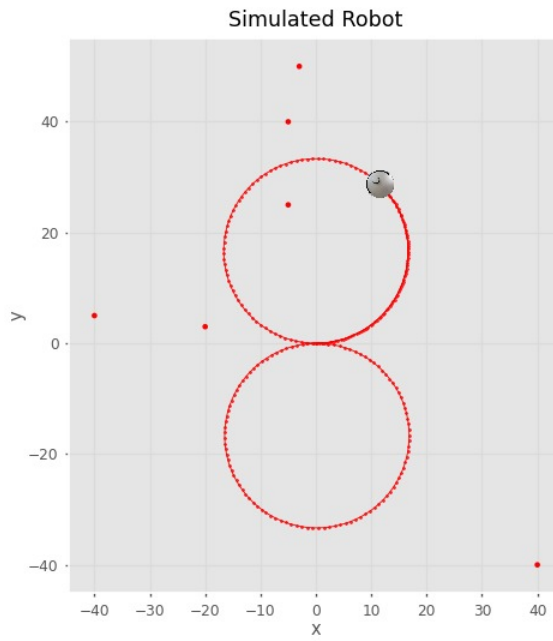


Figure 2: simulation for eight shaped trajectory.

3 Dead-Reckoning

Dead reckoning localization for a differential drive simulated robot estimates the robot's current position based on its previous location and the movements it has made. Using wheel encoder readings from its two independently controlled wheels, the robot calculates the distance traveled by each wheel and derives its overall displacement and change in orientation. The robot updates its position iterative using these calculations, represented in a 2D space as (x, y, ψ) .

In the code file DR 3DOFDifferentialDrive.py we completed the missing code for the following function: Localization and GetInput

- The **Localization** method takes the previous pose estimate and input from the wheel encoders to calculate the distances traveled by each wheel, the average distance, and the change in heading. It updates the robot's position and heading, incorporating Gaussian noise to simulate real-world sensor inaccuracies.
- The **GetInput** method retrieves the current readings from the wheel encoders, returning the input vector containing the left and right wheel readings. This class effectively enables the localization of a mobile robot by updating its state based on wheel movement while accounting for uncertainties from sensor noise.

In the output the simulated trajectory of Differential Drive Robot in circular path was obtained with some noise. Figure 3 shows the simulated result for circular path and the noise that cause the robot to deviate from the path.

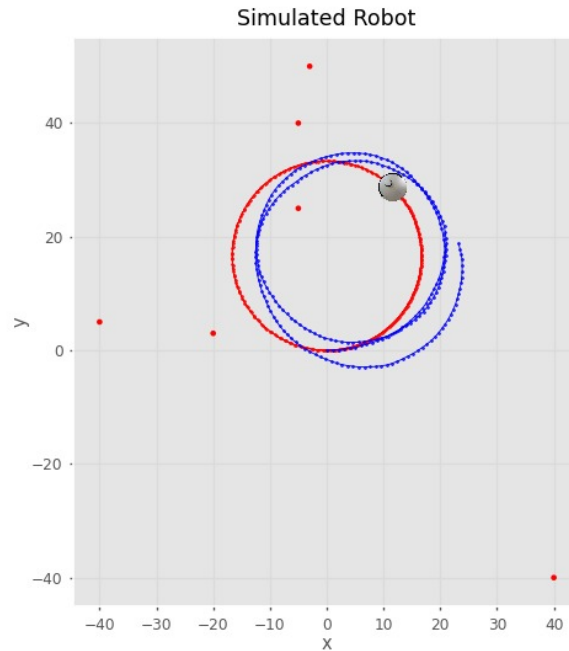


Figure 3: Figure 3:simulation for Dead Reckoning localization.

4 Issues

The simulation of a mobile robot faced several challenges related to pose updates and trajectory handling.

- One major issue was managing the impact of noise during pose updates, which could lead to significant drift in the robot's estimated trajectory. To address this, a Gaussian noise model was introduced, with specific standard deviations for the x-position and y-position.
- Dead reckoning inherently accumulates errors over time, leading to drift between the robot's true trajectory and its estimated path, particularly in longer simulations. To mitigate this, encoder noise was carefully simulated, and inputs were kept accurate within expected ranges by tuning noise levels.

5 Conclusions

In the lab we performed the simulation for Differential Drive Mobile Robot using our customized class from generic simulation code. The simulation was performed for two trajectory circular and eight paths. In the last part we simulated the Dead Reckoning localization with noise for the robot in circular path.

References

- [1] Pere Ridao. "Probabilistic:Robot Localization and mapping". Draft